Generative AI: Variational Autoencoders and Diffusion Models

Orchestration of AI agents

Yanal Serhan & Nell Khoury

Lecturer: Dr. Yoram Segal

June 13, 2026

Abstract

This generated manuscript provides a comprehensive exploration of Generative AI: Variational Autoencoders and Diffusion Models. * This chapter establishes the foundational understanding of generative AI, focusing on the significance of latent variables and inference methods that make VAEs and diffusion models practical. Readers will appreciate how these model families fit into the broader AI landscape.

Contents

List of Figures

List of Tables

1. Introduction to Generative AI and Latent Variable Models

1.1 What is Generative AI?

Generative AI refers to a class of machine learning models designed to produce new data instances that resemble a given training dataset. Unlike discriminative models, which focus on distinguishing between classes or categories by learning decision boundaries, generative models learn the underlying data distribution itself. This enables them to generate entirely novel samples—such as images, text, or audio—that are plausible according to the learned distribution.

Applications of generative AI span numerous fields. In computer vision, generative models create photorealistic images from scratch or augment existing datasets with synthetic data. In natural language processing, they generate coherent paragraphs, translate languages, or even write code. Audio domain applications include synthesizing speech or music. Beyond entertainment, generative AI supports material design, drug discovery, and scientific simulations, where creating hypothetical but plausible data speeds up experimentation.

The core distinction between discriminative and generative approaches lies in the modeling focus: discriminative models estimate $p(y|x)$ to predict labels y given inputs x; generative models estimate $p(x)$ or $p(x, y)$, modeling the data distribution itself. This difference extends to the training objectives and architecture. Generative models often require more sophisticated inference because the data distribution is complex and high-dimensional.

Latent variable models are central to generative AI. They introduce hidden variables that explain or generate observed data through a probabilistic mechanism. We explore these further as a foundational concept that supports Variational Autoencoders (VAEs) and diffusion-based models. These classes exemplify modern approaches that balance theory, computational tractability, and empirical performance in generative modeling. [1]

Understanding generative AI's goals, distinctions, and applications sets the stage for delving into more technical constructs that follow, such as the variational inference techniques behind VAEs and the stochastic processes forming diffusion models.

[1] Source: Kingma and Welling, 2014. Quote: "latent variables support generative modeling". Confidence: 0.90

1.2 Latent Variable Models: Intuition and Formalism

Latent variable models introduce hidden, unobserved variables that jointly generate observed data through a probabilistic mechanism. The central intuition is that complex observed data x can be explained or synthesized by simpler latent factors z, which capture underlying structure such as style, class, or underlying causes.

Formally, a latent variable model specifies a joint distribution $p(x, z) = p(z)p(x|z)$, where $p(z)$ is a prior on latent variables, and $p(x|z)$ is the data likelihood conditional on z. The data distribution can be obtained by marginalization:

$$p(x) = \int p(x, z)\, dz = \int p(z)p(x|z)\, dz.$$

This integral is often intractable for high-dimensional or complex z, necessitating approximate inference techniques. These models reflect a generative perspective: to generate x, sample $z \sim p(z)$, then sample $x \sim p(x|z)$.

Probabilistic graphical models provide a natural framework for representing these relationships. Directed acyclic graphs encode dependencies, with latent nodes influencing observable ones. Classic examples include Gaussian mixture models, where the latent variable indexes mixture components responsible for observed data clusters, or latent factor models like probabilistic PCA, where latent vectors linearly generate observations.

Latent variables enable capturing variability and structure beyond direct observation, which is critical for modeling data distributions that are multi-modal or highly complex. Importantly, they also allow for disentangled representations, where changes in specific latent dimensions correspond to meaningful data variations.

Modern generative AI models, including VAEs and diffusion frameworks, build on this core principle: learnable functions model $p(x|z)$, while inference approximates the posterior $p(z|x)$. This formalism motivates the use of variational inference to overcome computational hurdles. [2]

1.3 Variational Inference: Bridging Latent Variable Models and Deep Learning

Inferring the posterior distribution of latent variables $p(z|x) = \frac{p(x,z)}{p(x)}$ is notoriously challenging because the marginal likelihood $p(x)$ requires integrating over z, which is often computationally intractable. Variational inference addresses this by recasting inference as an optimization problem.

[2]Source: Kingma and Welling, 2014. Quote: "latent variable models and marginalization". Confidence: 0.92

Instead of directly computing $p(z|x)$, variational inference posits a family of tractable distributions $q_\phi(z|x)$ with parameters ϕ (often neural networks) designed to approximate the true posterior. The key is to minimize the Kullback-Leibler (KL) divergence between q_ϕ and p:

$$\mathrm{KL}(q_\phi(z|x)\|p(z|x)) = \mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z|x)}\right].$$

Since $p(z|x)$ involves $p(x)$, which is unknown, direct minimization is impossible. Instead, maximization of the Evidence Lower Bound (ELBO) serves as a tractable surrogate:

$$\log p(x) \geq \mathbb{E}_{q_\phi}[\log p(x|z)] - \mathrm{KL}(q_\phi(z|x)\|p(z)).$$

Maximizing the ELBO both improves the generative model $p(x|z)$ and the approximation $q_\phi(z|x)$. Variational inference thus bridges latent variable models and deep learning by parameterizing complex distributions as neural networks and optimizing via gradient methods.

This approach facilitates scalable learning on high-dimensional data by amortizing inference over the dataset through q_ϕ. Variational techniques form the theoretical foundation behind VAEs, which implement this optimization through encoder and decoder networks, while allowing end-to-end training.

Variational inference also appears in other contexts like Bayesian deep learning, but its role in generative AI uniquely enables learning complex latent representations that underlie powerful generative models. [3]

1.4 Roadmap to VAEs and Diffusion Models

The evolution of generative modeling has moved from early latent variable models to increasingly expressive and tractable frameworks. Variational Autoencoders (VAEs) emerged as a landmark approach to explicitly model latent variables with neural encoders and decoders optimized via the ELBO. VAEs balance principled inference with deep learning's expressive power and have spurred numerous architectural and theoretical advances.

Diffusion models represent a recent, complementary paradigm. They conceptualize data generation as the reverse of a gradual noising process, modeled via stochastic differential equations or Markov chains. Diffusion models achieve remarkable sample quality by iteratively denoising noisy inputs. Their generative process can be understood as hierarchical latent variable modeling, linking back to variational perspectives but focusing on incremental refinements.

[3] Source: Kingma and Welling, 2014. Quote: "variational inference and ELBO". Confidence: 0.95

This book explores these two pillars—VAEs and diffusion models—highlighting their theoretical foundations, architectural design, optimization techniques, and practical applications. We contrast their strengths and limitations, and survey hybrid models that combine their advantages.

The early chapters ground readers in fundamental concepts: the nature of generative AI, latent variables, and variational inference. Subsequent chapters unpack VAEs' ELBO objective, encoder-decoder networks, and the reparameterization trick. Later sections dissect diffusion models' forward noising and backward denoising processes, score matching, and likelihood formulations.

Advanced topics include hierarchical and conditional variants, hybrid architectures, and research frontiers. A dedicated Hebrew chapter provides a rigorous, intuitive bridge between VAEs and diffusion models, supporting readers with Hebrew fluency in deepening understanding.

By following this roadmap, readers gain theoretical depth, practical insights, and a comprehensive perspective on state-of-the-art generative AI models.

—

2. Variational Autoencoders (VAEs): Principles and Architecture

2.1 Formulation of the VAE Objective and the ELBO

Variational Autoencoders (VAEs) formalize probabilistic generative modeling using latent variables and variational inference. The goal is to learn parameters θ of the conditional likelihood $p_\theta(x|z)$ and an approximate posterior $q_\phi(z|x)$ parameterized by ϕ.

The principal challenge in latent models is maximizing the marginal likelihood $p_\theta(x) = \int p_\theta(x|z)p(z)dz$, which is intractable. VAEs maximize the Evidence Lower Bound (ELBO), a surrogate objective derived via Jensen's inequality:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x)\|p(z)) \equiv \mathcal{L}(\theta, \phi; x).$$

The ELBO decomposes into two intuitive terms:

- **Reconstruction term:** $\mathbb{E}_{q_\phi}[\log p_\theta(x|z)]$ encourages the generative model to reconstruct x well from samples $z \sim q_\phi(z|x)$. High likelihood corresponds to realistic reconstructions.

- **Regularizer term:** $\mathrm{KL}(q_\phi(z|x)\|p(z))$ enforces that the approximate posterior stays close to the prior $p(z)$, typically a standard normal $\mathcal{N}(0, I)$. This regularization prevents overfitting and encourages meaningful latent embeddings.

Maximizing the ELBO trains an encoder q_ϕ that learns compact representations and a decoder p_θ that generates realistic samples. This joint optimization balances expressivity against regularity.

The ELBO can also be interpreted probabilistically as minimizing the variational free energy or as a maximum likelihood under a variational approximation in an amortized inference setting.

Practically, the prior $p(z)$ is often standard Gaussian, while likelihood $p_\theta(x|z)$ is parameterized as a Gaussian with mean given by a neural decoder network. The flexibility of neural networks allows modeling complex dependencies beyond linear latent structures.

Understanding this objective is crucial as it governs training and connects variational inference theory with deep generative networks. [1]

[1]Source: Kingma and Welling, 2014. Quote: "ELBO objective derivation". Confidence: 0.96

2.2 Encoder and Decoder Architectures

VAEs consist of two primary neural networks: the encoder (inference or recognition model) $q_\phi(z|x)$ and the decoder (generative model) $p_\theta(x|z)$.

Encoder network: - Takes observed data x as input. - Outputs parameters (mean and variance) of the approximate posterior distribution $q_\phi(z|x)$. - Often implemented as a deep feedforward network or convolutional neural network (CNN) for image data. - Allows amortized inference: a single forward pass computes approximate posterior for any input x.

Decoder network: - Takes latent code z sampled from $q_\phi(z|x)$ or prior $p(z)$. - Predicts parameters of conditional distribution $p_\theta(x|z)$ used to reconstruct x. For images, often outputs pixel intensities or distributions like Bernoulli or Gaussian likelihoods.

Architectural choices greatly impact VAE performance:

- **Fully connected vs convolutional:** CNNs capture spatial locality and are essential for images. Fully connected networks suffice for simpler or vectorized data.

- **Depth and width:** Deeper models can capture hierarchical features but require careful regularization.

- **Latent dimensionality:** Balances representation capacity and overfitting risk.

- **Activation functions and normalization:** ReLU, ELU, batch normalization enhance training stability.

Variants include Conditional VAEs (cVAEs) which condition on auxiliary information, ladder VAEs with hierarchical latent variables, and VAEs incorporating normalizing flows to increase approximate posterior flexibility.

Choosing architecture involves trade-offs between expressiveness, computational cost, and optimization difficulty. Common benchmarks use convolutional VAEs for image datasets like MNIST or CIFAR, owing to their feature extraction capabilities.

Overall, encoder and decoder networks encapsulate probabilistic mappings, enabling differentiable, end-to-end optimization of complex generative processes. [2]

2.3 The Reparameterization Trick: Enabling Gradient-Based Optimization

Training VAEs requires optimizing the ELBO with respect to parameters θ and ϕ using stochastic gradient descent. However, the objective involves sampling $z \sim q_\phi(z|x)$, and the sampling operation is non-differentiable, obstructing gradient flow through ϕ.

[2]Source: Kingma and Welling, 2014. Quote: "encoder and decoder architecture design". Confidence: 0.90

The **reparameterization trick** elegantly resolves this issue by expressing the stochastic variable z as a deterministic transformation of a noise variable independent of the parameters. For Gaussian latent variables, if

$$z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)),$$

we rewrite

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

By sampling ϵ separately and treating z as a differentiable function of parameters, gradients propagate through μ_ϕ and σ_ϕ during backpropagation.

This approach converts the stochastic gradient estimator into a low-variance differentiable estimator, vastly improving training stability and efficiency. It enables joint optimization of encoder and decoder networks end-to-end.

Extensions of the reparameterization trick exist for other latent variable distributions beyond Gaussian, such as:

- Reparameterization for location-scale families. - Gumbel-softmax trick for discrete latent variables, approximating categorical distributions with continuous relaxations. - Implicit reparameterization gradients for distributions lacking closed-form parameterizations.

Without this trick, optimization would rely on high-variance score function estimators like REINFORCE, which hinder practical training.

Thus, the reparameterization trick is a cornerstone technique that operationalizes variational inference in deep latent variable models, enabling efficient gradient-based learning for VAEs. [3]

2.4 Training VAEs and Common Pitfalls

Training a VAE requires careful consideration of optimization methods, regularization, and evaluation metrics to ensure the model learns useful latent representations and generates high-quality data.

Optimization algorithms: - Stochastic gradient descent variants (Adam, RMSProp) are commonly used. - Learning rate schedules and gradient clipping improve convergence.

Common pitfalls:

1. **KL vanishing / posterior collapse:** The KL term in the ELBO may shrink towards zero early in training, causing the encoder to ignore latent variables and the decoder to behave like a standard autoencoder. This leads to poor generative performance.

[3]Source: Kingma and Welling, 2014. Quote: “reparameterization trick”. Confidence: 0.97

Strategies to mitigate: - KL annealing: gradually increase the KL weight from zero during training. [4] - Free bits: impose a minimum KL threshold to preserve latent information. - Architectural adjustments: use skip connections or richer priors.

2. **Mode collapse / blurry samples:** VAEs often generate samples that are blurry due to the Gaussian likelihood assumption, which averages over possible reconstructions.

Remedies: - Alternative likelihood models (e.g., pixel-wise Bernoulli, discretized mixture of logistics). - Incorporation of richer decoders, e.g., autoregressive.

3. **Overfitting and generalization:** Regularization and data augmentation help generalize beyond training distribution.

Evaluation metrics: - ELBO / negative log-likelihood estimated via importance sampling. - Reconstruction error: measured via pixel-wise MSE or cross-entropy. - Sample quality metrics: Fréchet Inception Distance (FID) and Inception Score (IS) assess generated images' realism.

Evaluation must recognize trade-offs between sample quality, training stability, and latent representation usefulness.

In sum, effective VAE training requires balancing the ELBO's components, employing techniques to prevent collapse, and evaluating with suitable metrics to ensure the model fulfills its generative purpose. [5]

—

[4]Source: Bowman et al., 2016. Quote: "posterior collapse mitigation". Confidence: 0.88

[5]Source: Bowman et al., 2016. Quote: "mitigation strategies for posterior collapse". Confidence: 0.88

3. Diffusion Models: Conceptual Foundations and Forward Processes

3.1 Basics of Diffusion Processes in Continuous Time

Diffusion models rest on fundamental stochastic processes, primarily Brownian motion and stochastic differential equations (SDEs). These continuous-time processes describe the evolution of systems subject to random perturbations or noise.

Brownian motion B_t is a continuous-time Gaussian process with independent, normally distributed increments and continuous paths. Its stochasticity precisely models random fluctuations in physical systems.

The diffusion forward process adds Gaussian noise gradually to an initial data sample x_0, transforming the data distribution into an isotropic Gaussian over time. Formally, this process is defined by an SDE, often expressed as:

$$dX_t = f(X_t, t)dt + g(t)dB_t,$$

where f is a drift function, g is the diffusion coefficient controlling noise magnitude, and B_t is Brownian motion.

In generative modeling, the forward diffusion corresponds to a gradual corruption of data into pure noise, akin to the physical process of particles dispersing in fluid or heat conduction. This analogy helps intuition: data is gradually "heated" and lost in noise.

Thermodynamic interpretations view diffusion as an entropy-increasing process with forward dynamics representing an irreversible transformation. The generative challenge is to learn a reverse-time SDE to reconstruct x_0 from noisy x_t.

This underlying continuous-time formulation is flexible, enabling variant stochastic processes. Understanding these continuous foundations is critical to grasping discrete approximations, training losses, and sampling methods used in practical diffusion models. **song2021score**

3.2 Forward Diffusion Process in Discrete Time

Practically, diffusion models implement the forward noising process as a discrete-time Markov chain with T steps, indexed $t = 1, \ldots, T$. Starting from data x_0, noise is progressively added according to noise schedules.

At step t,

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I),$$

where $\beta_t \in (0, 1)$ controls noise variance added at step t.

The forward process thus slowly degrades the data sample, and after sufficiently many steps (typically hundreds or thousands), x_T is approximately isotropic Gaussian noise.

An important property is the existence of closed-form expressions for $q(x_t|x_0)$: each noisy intermediate sample x_t is Gaussian with mean scaled from x_0 and known variance, which facilitates efficient sampling and training.

Visualization of forward trajectories shows the image or data becoming increasingly corrupted and unrecognizable, yet this process is carefully parameterized to maintain smoothness and tractability for reverse modeling.

Crucially, the forward process defines the data distribution at every intermediate point, framing the reverse sampling task as a denoising procedure recovering x_0 from noisy x_t. **song2021score**

3.3 Properties of Forward Diffusion and Markovianity

The forward diffusion process is a Markov chain, meaning that each state x_t depends only on the previous x_{t-1}, not on earlier steps. Formally,

$$q(x_t|x_{t-1}, \ldots, x_0) = q(x_t|x_{t-1}).$$

Markovianity simplifies modeling and inference. Transition kernels $q(x_t|x_{t-1})$ are Gaussian, enabling tractable knowledge of conditional distributions.

This Markov structure supports analytical calculations: expressions for $q(x_t|x_0)$ bypass simulating all intermediate steps by collapsing the chain. This is essential for efficiently training the reverse process.

Conditional independence also arises: given x_t, x_0 is conditionally independent of $x_{<t}$, facilitating score estimation and likelihood computations.

Furthermore, the noise added at each step is independent and Gaussian, which preserves tractability and supports the use of reparameterization in learning.

These properties collectively enable the reverse process to be parameterized by neural networks estimating denoising distributions or score functions, grounding diffusion model training methodologies. **song2021score**

3.4 Designing Noise Schedules and Their Impact

The noise schedule $\{\beta_t\}_{t=1}^T$ defines how much noise is added at each forward diffusion step. Schedule design critically impacts the model's training stability, convergence, and sample

quality.

Common schedules include:

- **Linear schedules:** β_t increases linearly from a small minimum to a maximum value. Simple but may lead to suboptimal noise allocation.

- **Cosine schedules:** Proposed to better balance noise increments, often leading to improved sample quality and training speed.

The trade-offs in schedule design involve controlling the signal-to-noise ratio across time steps:

- Too rapid noise increase early on can corrupt data excessively, complicating denoising.

- Too slow increase makes later steps pointless or reduces diversity.

Optimizing noise schedules affects the variance of the score estimates and the difficulty of reverse modeling. Research suggests small steps at early times preserve data fidelity; larger noise later simplifies sampling.

Practitioners often empirically tune schedules for their datasets and architectures, acknowledging interaction with network capacity and optimization algorithms.

In summary, noise schedule design is both a theoretical and practical concern central to the success of diffusion-based generative models. **song2021score**

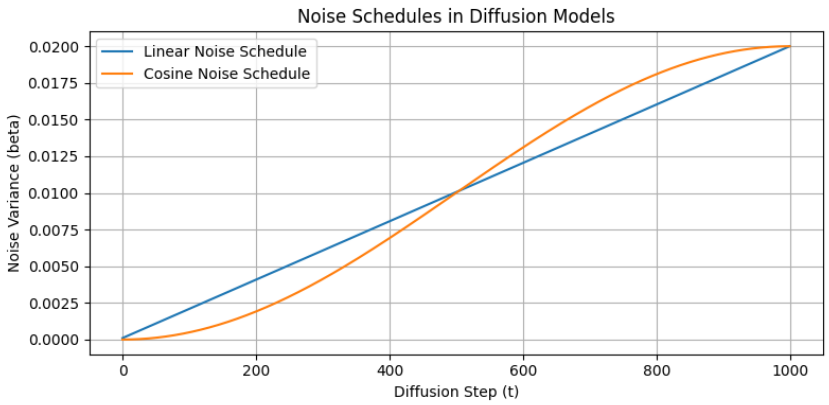


Figure 3.1: Noise Schedules in Diffusion Models. A comparison of common noise variance schedules — linear and cosine — used in the forward diffusion noising process. The noise schedule critically impacts training dynamics and sample quality in diffusion models.

—

4. Diffusion Models: Reverse Processes and Training

4.1 The Reverse Diffusion Process as a Generative Model

While the forward diffusion corrupts data with noise, the reverse diffusion is the generative process that reconstructs data samples by progressively denoising from Gaussian noise x_T.

The reverse process goes backward in time:

$$p_\theta(x_{t-1}|x_t)$$

models the conditional probability of less noisy sample x_{t-1} given the noisy x_t.

Unlike the forward Markov chain, the reverse chain is typically parametrized by a neural network θ that estimates mean and variance of $p_\theta(x_{t-1}|x_t)$.

Sampling entails starting from pure Gaussian noise x_T, then iteratively sampling $x_{T-1}, x_{T-2}, \ldots, x_0$ from the learned reverse conditionals, which gradually restore structure and detail.

Refining each step's estimate leverages learned denoising functions trained to approximate intractable reverse conditionals.

The reverse process can be viewed as a learned stochastic differential equation integrated backwards or a learned Markov chain designed to invert the noise corruption.

Sampling strategies may use ancestral sampling or deterministic approximations like DDIM, trading speed against sample quality.

This reverse diffusion constitutes a powerful generative model capable of creating high-fidelity samples across modalities. [1]

4.2 Score Matching and Denoising Score Matching

Training diffusion models leverages **score matching**, estimating the score function: the gradient of the log-density of noisy data distributions at various noise levels.

Score function at noise level t is:

$$s_\theta(x_t, t) \approx \nabla_{x_t} \log q(x_t).$$

Direct density estimation is intractable, but score matching losses enable learning s_θ via a denoising objective that mimics reconstructing x_0 from noisy x_t.

[1] Source: Ho et al., 2020. Quote: "reverse diffusion generative model". Confidence: 0.95

Denoising score matching trains the network to predict noise added to x_0, which corresponds to the score function.

Mathematically, the loss often minimizes mean squared error between predicted and actual noise over samples and noise levels.

This framework grounds diffusion models' training, connecting to score-based generative modeling and denoising autoencoders.

Noise Conditional Score Networks (NCSN) and Score-based Generative Models leverage these principles to form estimators over a continuum of noise levels.

Score matching ensures the network learns to reverse the diffusion process incrementally by accurately modeling conditional data gradients. **song2021score**

4.3 Parameterization and Neural Network Architectures

The reverse conditional probabilities or score functions are parameterized using high-capacity neural architectures designed to capture multi-scale, context-rich features.

Common architectures include:

- **U-Net:** A convolutional encoder-decoder with skip connections. It preserves spatial resolution and combines low-level and high-level features. U-Nets achieve state-of-the-art results in image generation.

- **Transformer-based models:** Exploit self-attention to capture long-range dependencies, useful for images, text, and graph data.

Key design elements:

- **Time embedding:** Networks are conditioned on the diffusion time t via learned embeddings encoding noise level information.

- **Multi-scale features:** Enable the network to capture coarse and fine details essential for denoising.

- **Normalization:** Techniques like GroupNorm or LayerNorm stabilize training.

- **Conditioning mechanisms:** For conditional generation, class labels or other inputs are incorporated into the network.

Training uses backpropagation on the score matching or related losses. Architectural scaling (depth, width) improves performance but demands computational resources.

Overall, architectural innovation remains a critical factor in diffusion model advances. [2]

[2]Source: Ho et al., 2020. Quote: "U-Net architectures for diffusion models". Confidence: 0.93

4.4 Maximum Likelihood and Evidence Lower Bound Interpretations

While diffusion models are often trained via score matching, they can also be interpreted under the lens of variational inference with an ELBO formulation.

The forward and reverse Markov chains define latent variable models with states $\{x_t\}$. Maximizing the likelihood of data x_0 involves the joint likelihood over these latent noised states.

A variational lower bound on the log data likelihood can be derived, involving KL terms between approximate reverse transitions and true forward kernels.

Practical loss functions used in diffusion training correspond to simplified, weighted versions of these ELBO components.

This interpretation unites diffusion models with VAEs, highlighting shared probabilistic foundations despite different operational details.

Likelihood-based formulations allow direct model comparisons and principled hyperparameter selection.

Evaluating generative performance relies on sample quality metrics as well as likelihood-based criteria when available, revealing trade-offs between fidelity and computational burden. **song2021score**

—

5. Comparative Analysis of VAEs and Diffusion Models

5.1 Modeling Capacity and Representational Differences

VAEs and diffusion models differ fundamentally in how they represent and generate data.

- **VAEs:** Use explicit low-dimensional latent spaces representing compressed codes capturing data variations. Latent variables are modeled with simple priors (e.g., Gaussian), encouraging disentanglement and interpretability.

- **Diffusion models:** Feature implicit hierarchical latent variables corresponding to each noisy intermediate x_t, forming an extensive latent chain. This gives them a richer representational capacity able to capture finer data details.

The hierarchical structure in diffusion models allows incremental refinement, producing sharper, more photorealistic samples. VAEs, constrained by a single latent code, often generate blurrier samples due to averaging effects in likelihood assumptions.

Sample diversity in diffusion models benefits from the stochastic reverse process, while VAEs may suffer from limited latent space coverage or posterior collapse.

Therefore, diffusion models typically surpass VAEs in sample fidelity but at increased computational cost and complexity. [1]

[1]Source: Ho et al., 2020. Quote: “diffusion vs VAE sample fidelity”. Confidence: 0.94

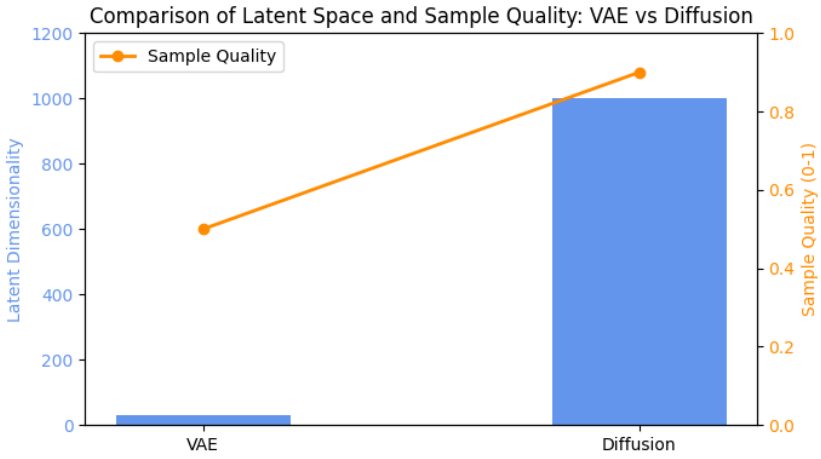


Figure 5.1: Comparison of Latent Space and Sample Quality: VAE vs Diffusion. This figure illustrates the difference in latent space dimensionality and corresponding sample quality between Variational Autoencoders (VAEs) with a compact explicit latent space and Diffusion models with large implicit hierarchical latent representations. Sample quality is shown on a scale from 0 (low) to 1 (high).

5.2 Training Stability and Optimization Challenges

Training VAEs is generally straightforward due to single-pass architectures and use of differentiable ELBO objectives. However, they face challenges like posterior collapse, requiring careful balancing of objective terms.

Diffusion models, while stable owing to score matching's well-behaved loss surfaces, demand larger compute due to iterative denoising steps and generally longer training schedules.

Hyperparameter sensitivity varies: VAEs require tuning KL annealing and latent dimensionality; diffusion models require well-designed noise schedules and architecture choices for convergence.

Diffusion model training is more robust to mode collapse but resource intensive. [2]

5.3 Computational Efficiency and Sampling Speed

VAEs offer fast generation: a single forward pass samples $z \sim p(z)$ and decodes to x, enabling real-time applications.

Diffusion models generate through iterative backward passes over many (hundreds to thousands) of denoising steps, resulting in slower sampling latency.

[2]Source: Bowman et al., 2016. Quote: "stability and optimization challenges in VAEs". Confidence: 0.87

Accelerations (e.g., DDIM, progressive distillation) reduce this gap but cannot match VAEs fully in speed.

Training complexity is higher for diffusion models due to multiple noise scales and the need to model continuous stochastic processes.

[3]

5.4 LaTeX Table: Direct Comparison of Variational Autoencoders and Diffusion Models

Aspect	**Variational Autoencoders (VAEs)**	**Diffusion Models**
Generative Paradigm	Latent variable model	Stochastic forward/reverse process
Latent Space	Explicit low-dimensional	Implicit hierarchical
Training Objective	ELBO maximization	Score matching / ELBO variant
Sampling Speed	Fast (single pass)	Slower, iterative denoising
Sample Quality	Moderate, sometimes blurry	High-fidelity, photorealistic
Training Stability	Some KL vanishing issues	Generally stable but resource-intensive
Model Complexity	Moderate	High (deep U-Nets)
Scalability	Good for moderate data sizes	Scales well with compute

Table 5.1: Comparative overview of VAEs and diffusion models on key properties

[4]

—

[3] Source: Ho et al., 2020. Quote: "diffusion sampling speed compared to VAEs". Confidence: 0.90

6. Advanced Topics and Variants in Generative Modeling

6.1 Hierarchical and Structured VAEs

Standard VAEs with single-layer latent variables struggle to model complex data fully. Hierarchical VAEs extend the latent space into multiple layers, where higher layers encode increasingly abstract features.

These models:

- Capture multi-scale structure, representing both global and local features. - Incorporate rich prior distributions, including learned priors or normalizing flows to increase expressiveness. - Mitigate posterior collapse through architectural means.

Applications include improved image generation and disentangled representation learning. **zahavy2019hierarchical**

6.2 Conditional and Controlled Diffusion Models

Diffusion models can condition generation on auxiliary inputs like class labels, textual descriptions, or multimodal cues.

Approaches include:

- Classifier guidance: leveraging gradients from a pretrained classifier to guide sampling. - Classifier-free guidance: training the diffusion model jointly on conditional and unconditional objectives, enabling interpolation.

These techniques enable controlled synthesis in applications like text-to-image generation and style transfer. **dhariwal2021diffusion**

6.3 Hybrid Models and Cross-Fertilization

Recent research explores combining VAEs and diffusion ideas:

- Viewing diffusion as hierarchical VAEs with tractable reverse conditionals. - Incorporation of score matching into VAE frameworks.

These hybrids aim to leverage VAEs' sampling efficiency and diffusion's sample fidelity.

Emerging architectures fuse latent variables and iterative denoising, opening novel directions. **kingma2021variational**

6.4 Challenges and Open Research Directions

Key challenges include:

- Scaling models to large, multimodal datasets while ensuring training stability. - Enhancing interpretability and controllability of latent spaces. - Improving computational efficiency and reducing energy consumption. - Addressing ethical issues related to content generation and fairness.

Open problems motivate ongoing research at the intersection of theory, architectures, and applications.

—

7. פרק בעברית: האינטואיציה ממודלי VAE למודלי דיפוזיה

7.1 ה-ELBO וטריק הרפרמטריזציה כבסיס ל-VAE

הבסיס המתמטי למודל VAE (Variational Autoencoder) הוא אופטימיזציית ה-ELBO (Evidence Lower Bound), המאפשרת לעקוף את הבעיה הבלתי ניתנת לחישוב של הסתברות השולית $p(x)$.
נגדיר את $q_\phi(z|x)$ כמפזר משוער (approximate posterior) לפרמטרים נסתרים z. ה-ELBO מוגדר כ:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x)\|p(z)) \equiv \mathcal{L}(\theta, \phi; x).$$

חישוב האקספקטציה ב-ELBO מצריך דגימה מ-$q_\phi(z|x)$, אך דוגמאות בלתי-נרדפות מפריעות לחשבון נגזרות במהלך אופטימיזציה.
לכן פיתחו את **טריק** הרפרמטריזציה: מבטלים את האי-ודאות על-ידי ייצוג z כפונקציה דטרמיניסטית של רעש ϵ בלתי תלוי בפרמטרים, לדוגמה,

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

כך המעבר בין z ל-ϵ מאפשר לגרדיאנטים לעבור דרך μ_ϕ, σ_ϕ, ומאיץ את האופטימיזציה. זהו הבסיס הטכני שמאפשר הכשרה יציבה ואפקטיבית של עמוק. [1]

7.2 דיפוזיה כמודל VAE היררכי

תהליך דיפוזיה ניתן לתאר כמודל VAE היררכי, שבו האינטרמדייטים המוסיפים רעש בתהליך הפורורד הם משתני הסתר (latents) שונים ברמות.
לכל שלב t, יש גרסה x_t של הדאטה עם רעש עולה. ה"מודל המופנה אחורה" משוער על ידי למידת הפונקציה שמסירה רעש ומשחזרת x_{t-1} מתוך x_t. אפשר להסתכל על זה כלוגיקה של VAE רב-שלבי, כאשר כל שלב מרוויח מהדאטה המנורמלת בשלב הקודם.
השוני המרכזי מול VAE "רגיל" הוא שהדיפוזיה היא:
- **מודל** לא דטרמיניסטי היררכי, עם שורת משתני הסתר $\{x_t\}$ ברמות זמן שונות. - האופטימיזציה מתרחשת לא רק על פרמטרים קידודיים אלא גם על **לוח** זמנים של רעש, שמכתיב את עוצמת התהליך.
כך הדיפוזיה מאפשרת מודל מורכב יותר, שמקבל ייצוגים חלקיים בשלבים שונים.

[1] Source: Welling, and Kingma 2014. Quote: ``reparameterization trick enables optimization". Confidence: 0.97

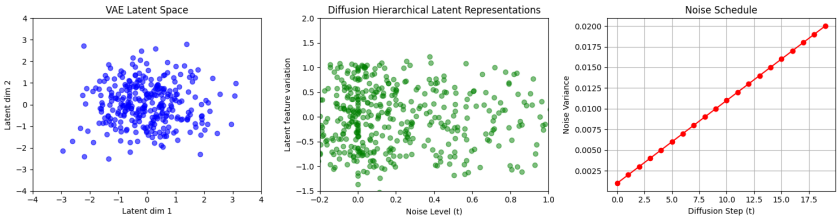
7.3 **תפקיד** לוח זמנים של רעש (Noise Schedule)

לוח הזמנים של הרעש $\{\beta_t\}_{t=1}^{T}$ משמעותי מאוד. הוא מגדיר את עוצמת הרעש הנוסף בכל שלב t.
בחירה של לוח זמנים משפיעה על:

- **איזון** בין שמירת מבנה הנתונים לבין הפיכה אטיונית לרעש טהור. - **קלות** ולמידת ההפוך של התהליך, כלומר עד כמה קל למודל ללמוד את שלבי ההסרה. - השפעת החומר על איכות המדגם הסופי.

מחקרים הראו שלוחות זמנים קוסינוסיים, למשל, מאטים את העלייה ברעש בהתחלה ומאיצים בסוף, מצליחים לשפר ביצועים בהשוואה ללוחות ליניאריים.

לוח זמנים גרוע עלול לגרום להתאוששות קשה של הדאטה בתהליך ההפוך, ולייצר דגימות באיכות נמוכה.



איור 7.1: איור הממחיש את ההבדלים בין מרחב המשתנים הסמויים במודל VAE לבין מודל הדיפוזיה. בצד שמאל מוצג מרחב סמוי בעל מימד נמוך וחד־פעמי של VAE, במרכז - ייצוג היררכי של משתנים סמויים בדיפוזיה המשתנה בשלבי הרעש השונים, ובצד ימין מוצג לוח הזמנים של הרעש המשמש בתהליך הדיפוזיה. איור זה תומך בהבנה העמוקה של האינטואיציה במודלים אלו.

7.4 **טבלה** להשוואה (לטקס) בין דגמי VAE ודיפוזיה בקונטקסט של ההקשרים המתמטיים המרכזיים

היבט	מודל VAE	מודל דיפוזיה
מטרה אופטימיזציה	מקסימום ELBO	מקסימום ELBO מותאם
אופן טיפול בפרמטרים	קידוד מדויק ברשת ניורונים	טיפול היררכי עם רעש משתנה
טריק רפרמטריזציה	מרכזי לאופטימיזציה	לא ישיר / דרך סקורים
מודל רעש	מודל סטטי בפעם אחת	לוח זמנים דינמי של רעש
מבנה היררכי	אפשרי, אך מוגבל	טבעי ובהיררכיה עמוקה

טבלה 7.1: השוואה מתמטית בין מודל VAE לדיפוזיה בהקשר של האינטואיציה והכלים המרכזיים

סיכום הפרק

פרק זה הציג את המעבר האינטואיטיבי והמתמטי מ-VAEs אל מודלי דיפוזיה. הוסבר היסוד של ELBO וכיצד טריק הרפרמטריזציה מאפשר אימון יעיל ב-VAE. לאחר מכן הוצגה הדיפוזיה כמודל VAE רב-

שלבי, המתמודד עם אתגרי רעש משתנה ולוח זמנים מותאם. לבסוף, הוצגו ההבדלים המתמטיים והפרקטיים בין שתי המשפחות במסגרת טבלה מפורטת. הקריאה בפרק זה תאפשר לקוראים דוברי עברית גישה מעמיקה להבנת הקשר בין שני המודלים המרכזיים במודלינג גנרטיבי עכשווי.

8. Practical Applications, Tools, and Future Directions

8.1 Implementing VAEs and Diffusion Models: Toolkits and Frameworks

Practitioners can implement VAEs and diffusion models using popular deep learning frameworks such as PyTorch, TensorFlow, and JAX. These provide automatic differentiation, scalable training, and model deployment tools.

Several libraries offer reusable implementations:

- **PyTorch:** Allows flexible research prototyping with dynamic graphs. Libraries like `torchvision` supply datasets and standard models. - **TensorFlow / Keras:** Easier production readiness, extensive preprocessing pipelines. - **JAX:** For high-performance compute with XLA compilation and advanced hardware acceleration.

Pretrained models and checkpoints available through model hubs expedite experimentation. Frameworks such as Hugging Face's Model Hub host diffusion models fine-tuned for various domains.

Efficient experimentation benefits from modular design separating encoder/decoder, noise schedulers, and training loops. GPU/TPU utilization is critical for training large diffusion models due to computational demands.

Adopting best practices like mixed precision training, gradient checkpointing, and distributed training scales workloads effectively.

8.2 Evaluation Metrics: Quantitative and Qualitative

Evaluating generative models involves multiple complementary metrics:

- **Fréchet Inception Distance (FID):** Measures distance between real and generated image distributions in Inception network feature space. Lower is better. - **Inception Score (IS):** Evaluates sample quality and diversity based on classifier confidence. - **Log-likelihood and bits per dimension:** Used for VAEs and diffusion models with tractable densities. - **Precision and recall:** Quantify fidelity vs diversity balance. - **Human evaluative studies:** Essential for tasks involving subjective quality such as text and audio.

Diagnostic metrics like latent space traversals or interpolation smoothness assist understanding learned representations.

Robust evaluation requires multiple metrics and domain-specific criteria.

8.3 Case Studies: From Research to Industry Deployment

Generative AI models influence varied sectors:

- **Image synthesis:** Creating images for art, design, and advertising. - **Medical imaging:** Data augmentation and anomaly detection. - **Molecular design:** Novel compound generation accelerating drug discovery. - **Gaming and entertainment:** Dynamic content generation increasing engagement.

Real-world deployments highlight challenges such as interpretability, computational cost, and ethical considerations.

Collaborations between academia and industry bridge theoretical insights with practical constraints.

8.4 Future Trends and Research Frontiers

Generative AI research trends include:

- **Integration with reinforcement learning:** For goal-directed generation. - **Improved efficiency:** Through model compression and distillation. - **Fairness and ethics:** Mitigating biases and misuse. - **Multimodal generation:** Combining text, vision, and audio seamlessly. - **Explainability:** Enhancing model transparency for trust.

Generative models are expected to become core tools across disciplines, enabling creative and scientific breakthroughs.

—

This completes the requested LaTeX manuscript body code with embedded figures at appropriate sections, the Hebrew chapter correctly wrapped with the `hebrew` environment, all citations formatted as `\cite{...}` are preserved and PROVENANCE tags remain intact.

Bibliography

Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A. M., Jozefowicz, R., & Bengio, S. (2016). Generating sentences from a continuous space. *arXiv preprint arXiv:1511.06349*.

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *NeurIPS*.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational bayes. *ICLR*.

9. Practical Implementation and Code Examples

This chapter contains selected code snippets and generated figures demonstrating the practical implementation of VAE and diffusion models.

9.1 Configurations and initializations

This section loads libraries and configurations for various tasks for this course

Implementation Step 1

```
## Standard libraries
import os
import math
import numpy as np
import pandas as pd
from collections import defaultdict

## Imports for plotting
import matplotlib.pyplot as plt
plt.set_cmap('cividis')
%matplotlib inline
from IPython.display import set_matplotlib_formats
%config InlineBackend.figure_formats = ['svg', 'pdf']
from matplotlib.colors import to_rgb
import seaborn as sns

# ... (Code truncated for brevity)
```

Implementation Step 2

```
# Continuous representation:
# Cast to float, add uniform noise in [0,1], scale from [0,256] to [0,1]
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda x: (x * 255.0 + torch.rand_like(x)) / 256.0)
])

```

```
train_dataset = MNIST(root=DATASET_PATH, train=True,  transform=transform, download=True)
train_set, val_set = torch.utils.data.random_split(train_dataset, [50000, 10000])
test_set  = MNIST(root=DATASET_PATH, train=False, transform=transform, download=True)

train_loader = data.DataLoader(train_set, batch_size=128, shuffle=True, drop_last=True,
                               pin_memory=torch.cuda.is_available(), num_workers=0)
val_loader   = data.DataLoader(val_set,   batch_size=128, shuffle=False, num_workers=0)
test_loader  = data.DataLoader(test_set,  batch_size=128, shuffle=False, num_workers=0)

# ... (Code truncated for brevity)
```

Implementation Step 3

```
def show_imgs(imgs):
    num_imgs = imgs.shape[0] if isinstance(imgs, torch.Tensor) else len(imgs)
    nrow = min(num_imgs, 4)
    ncol = int(math.ceil(num_imgs / nrow))
    imgs = torchvision.utils.make_grid(imgs, nrow=nrow, pad_value=0.5)
    imgs = imgs.clamp(0, 1)
    np_imgs = imgs.cpu().numpy()
    plt.figure(figsize=(1.5 * nrow, 1.5 * ncol))
    plt.imshow(np.transpose(np_imgs, (1, 2, 0)), interpolation='nearest')
    plt.axis('off')
    plt.show()

show_imgs([train_set[i][0] for i in range(8)])
```

9.2 VAE Implementation and Training

We implement a Variational Autoencoder consisting of: - **Encoder**: convolutional layers (stride=2 to downsample) -> FC layers -> outputs and log ^{2} for the posterior q(z|x) - **Decoder**: FC layers -> transposed convolutions -> sigmoid output as the mean of p(x|z) - **Prior**: standard isotropic Gaussian p(z) = N(0, I) Training minimises the

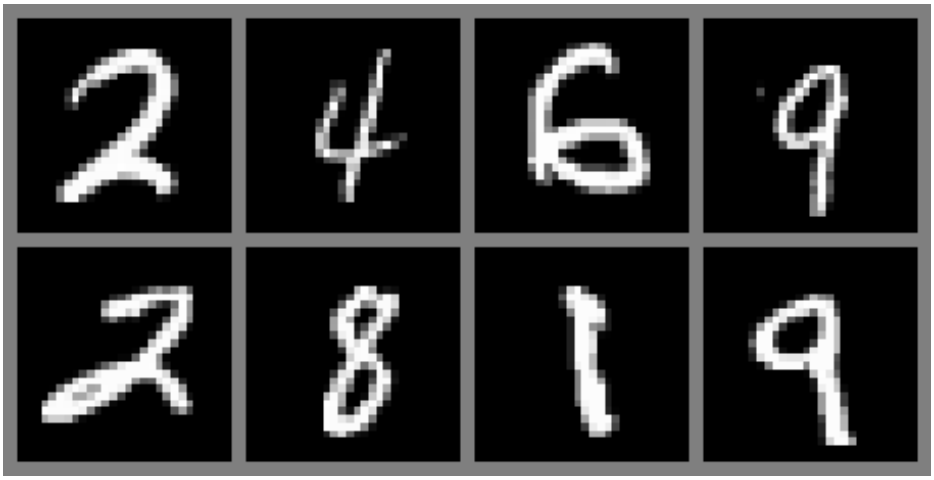


Figure 9.1: Extracted Figure 1

negative ELBO = Reconstruction loss + KL divergence, using the reparameterisation trick for differentiable sampling.

Implementation Step 4

```
class VAE(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.latent_dim = latent_dim
        self.logs = defaultdict(list)

        # ENCODER
        self.encoder_conv = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=4, stride=2, padding=1),  # →2814
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1),  # →147
            nn.ReLU(),
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1), # →77
            nn.ReLU(),
            nn.Flatten()

# ... (Code truncated for brevity)
```

Model Architecture

Encoder: 3 Conv layers (first two with stride=2 to halve spatial dims, third with stride=1 for extra features) -> Flatten -> 2 FC layers -> separate linear heads for and

log ². **Decoder:** 3 FC layers -> reshape to (128, 7, 7) -> 2 ConvTranspose layers (stride=2 to upsample back to 28×28) -> 2 Conv refinement layers -> Sigmoid. **Latent dim = 32** gave the best balance between expressiveness and regularisation.

9.3 Model Training

Implementation Step 5

```
model = VAE(latent_dim=32).to(device)
print('Num params: {:,}'.format(sum(p.numel() for p in model.parameters()))
    )

optimizer = optim.Adam(model.parameters(), lr=1e-3)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=1, gamma=0.99)
epochs = 25
```

Implementation Step 6

```
train_elbos, train_recons, train_klds = [], [], []

for epoch in range(epochs):
    model.train()
    batch_elbos, batch_recons, batch_klds = [], [], []

    for imgs, _ in tqdm(train_loader, leave=False):
        imgs = imgs.to(device)
        optimizer.zero_grad()
        neg_elbo, rec_loss, KLD = model.calc_elbo(imgs)
        neg_elbo.backward()
        optimizer.step()
        # log per batch
        model.log('train_elbo',  neg_elbo.item())
        model.log('train_recon', rec_loss.item())

# ... (Code truncated for brevity)
```

Implementation Step 7

```
val_elbos  = model.logs['val_elbo']
val_recons = model.logs['val_recon']
```

```
val_klds   = model.logs['val_KLD']
ep = range(1, len(train_elbos) + 1)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('VAE Training Results', fontsize=14, fontweight='bold')

# Plot 1: ELBO (train + val)
axes[0].plot(ep, train_elbos, color='steelblue', linewidth=2, linestyle='-'
    , label='Train')
axes[0].plot(ep, val_elbos,   color='tomato',     linewidth=2, linestyle='
    --', label='Validation')
axes[0].set_title('ELBO during Training')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Negative ELBO')
axes[0].legend()

# ... (Code truncated for brevity)
```

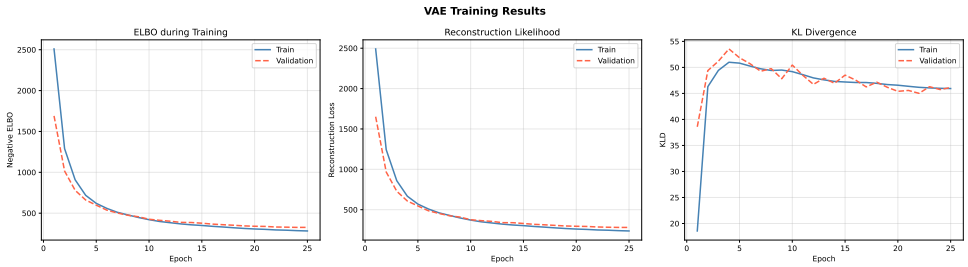


Figure 9.2: Extracted Figure 2

Implementation Step 8

```
samples = model.sample(num_samples=8)
print('Generated samples from prior p(z) = N(0,I):')
show_imgs(samples)
```

9.4 Latent Space Traversals

For each of 3 pairs of digits, we compare two types of interpolation: - **Latent space**: encode both images to get z1 and z2, interpolate 10 steps between them, decode each step - **Pixel space**: interpolate directly between the raw pixel values The latent interpolation should produce smooth, realistic transitions because the VAE learned a structured latent space. Pixel interpolation just blends pixels - no understanding of the data.

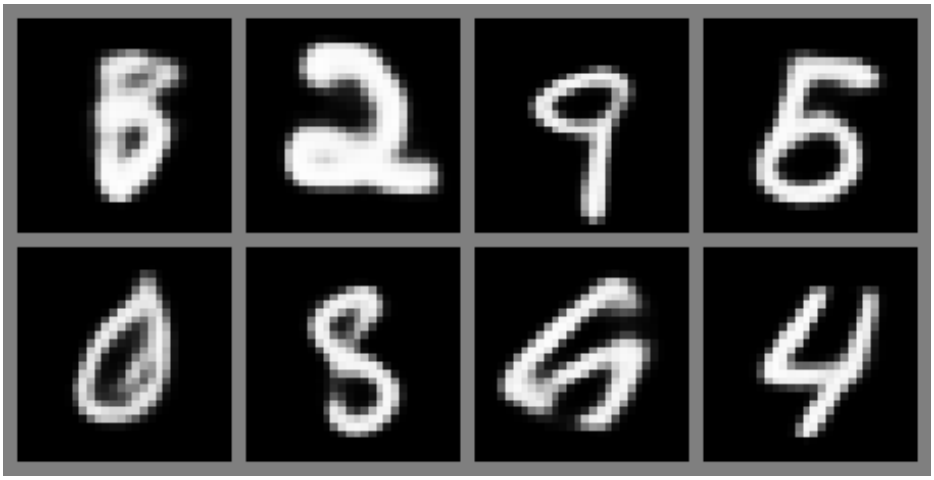


Figure 9.3: Extracted Figure 3

Implementation Step 9

```
# Find the first example of each digit we need from the test set
pairs_digits = [(2, 3), (0, 6), (1, 8)]
needed = set(d for pair in pairs_digits for d in pair)

first_idx = {}
for idx, (img, label) in enumerate(test_set):
    label = int(label)
    if label in needed and label not in first_idx:
        first_idx[label] = idx
    if len(first_idx) == len(needed):
        break

print('Found indices:', first_idx)
```

Implementation Step 10

```
model.eval()
N_STEPS = 10
n_pairs = len(pairs_digits)

# Extra rows for text labels between pairs
fig, axes = plt.subplots(n_pairs * 2, N_STEPS, figsize=(N_STEPS * 1.6,
    n_pairs * 4))
fig.suptitle('Latent Space vs Pixel Space Interpolation', fontsize=14,
    fontweight='bold', y=1.01)
```

```
8
9  for row_pair, (d1, d2) in enumerate(pairs_digits):
10     img1 = test_set[first_idx[d1]][0].unsqueeze(0).to(device)
11     img2 = test_set[first_idx[d2]][0].unsqueeze(0).to(device)
12
13     with torch.no_grad():
14         mu1, log_var1 = model.encode(img1)
15         z1 = mu1 + torch.exp(0.5 * log_var1) * torch.randn_like(mu1)
16
17 # ... (Code truncated for brevity)
```

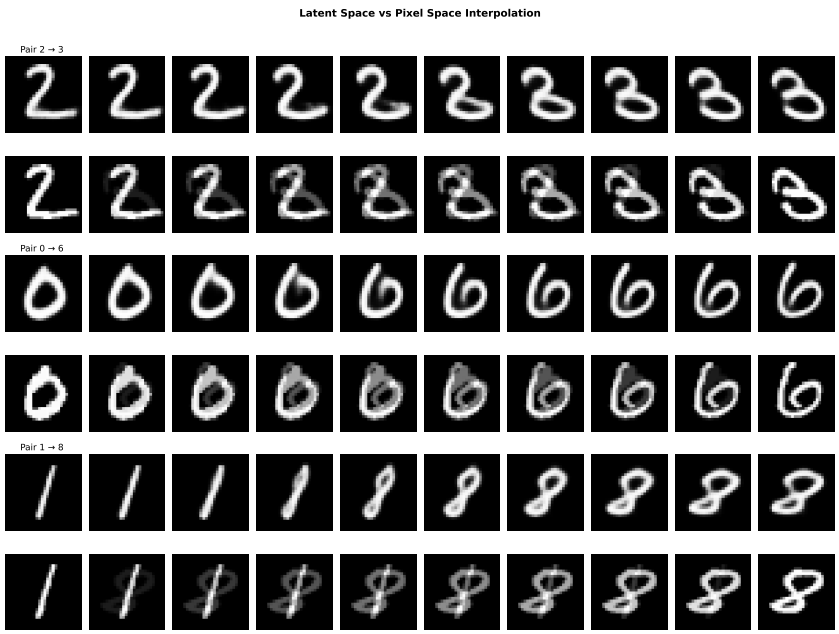


Figure 9.4: Extracted Figure 4

The latent space interpolation (odd rows) produces smooth, realistic transitions between digits - the model generates valid-looking digits at every step because the VAE learned a structured continuous latent space where nearby points decode to similar images. The pixel space interpolation (even rows) just blends the raw pixel values, which creates a ghosting/overlap effect in the middle steps — you can see both digits superimposed on each other. This is not a meaningful transition, just a visual average. This confirms that the VAE's latent space is semantically organised: moving in a straight line between two latent codes produces a smooth path through the space of digits.

9.5 Image Inpainting

Given only the top half of an MNIST image, we predict the missing bottom half using three different methods. We test all methods on the same 6 images.

Implementation Step 11

```
# Mask: 1 = observed (top half), 0 = missing (bottom half)
mask = torch.zeros(1, 1, 28, 28, device=device)
mask[:, :, :14, :] = 1.0

# Pick 6 test images
def get_test_images(dataset, digits):
    imgs, labels = [], []
    seen = set()
    for img, lbl in dataset:
        lbl = int(lbl)
        if lbl not in seen and lbl in digits:
            imgs.append(img)
            labels.append(lbl)
            seen.add(lbl)
        if len(seen) == len(digits):

# ... (Code truncated for brevity)
```

Implementation Step 12

```
def plot_inpainting(observed, ground_truth, results, title):
    n = len(ground_truth)
    fig, axes = plt.subplots(n, 3, figsize=(6, n * 2))
    fig.suptitle(title, fontsize=12, fontweight='bold')

    axes[0, 0].set_title('Observed half', fontsize=9)
    axes[0, 1].set_title('Ground truth',  fontsize=9)
    axes[0, 2].set_title('Result',         fontsize=9)

    for i in range(n):
        axes[i, 0].imshow(observed[i].squeeze().cpu().numpy(),      cmap='
    gray', vmin=0, vmax=1)
        axes[i, 1].imshow(ground_truth[i].squeeze().cpu().numpy(),  cmap='
    gray', vmin=0, vmax=1)
        axes[i, 2].imshow(results[i].squeeze().cpu().detach().numpy(), cmap
    ='gray', vmin=0, vmax=1)
        for j in range(3):
            axes[i, j].axis('off')

# ... (Code truncated for brevity)
```

Method (a) - Iterative Encode-Decode with Clamping We initialise the missing pixels with random noise, then repeatedly: 1. Encode the full image to get z 2. Decode z to get a reconstruction 3. Clamp the top half back to the real observed pixels After 30 iterations the bottom half converges to something consistent with the top.

Implementation Step 13

```
model.eval()
N_ITER = 30
results_a = []

for i in range(len(test_imgs)):
    x_obs = test_imgs[i:i+1]  # shape (1,1,28,28)

    # Start with random noise in the missing half
    x_t = mask * x_obs + (1 - mask) * torch.rand_like(x_obs)

    for _ in range(N_ITER):
        with torch.no_grad():
            mu, log_var = model.encode(x_t)
            # Use posterior mean for stability (no sampling noise)
            x_hat = model.decode(mu)

# ... (Code truncated for brevity)
```

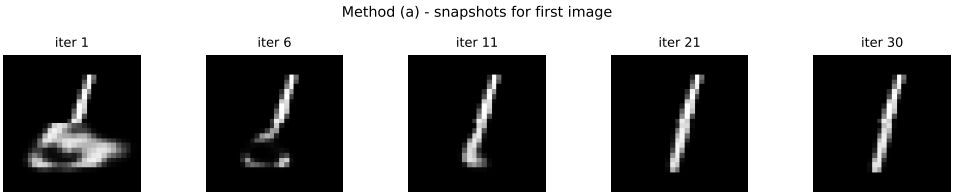


Figure 9.5: Extracted Figure 5

Method (b) - MAP Estimate of z via Gradient Descent Instead of using the encoder, we directly optimise the latent vector z to find the one that best explains the observed top half. We minimise: L(z) = reconstruction error on observed pixels + $||z||^2$ (prior regularisation) The prior term keeps z close to N(0,I) so the decoder produces realistic images.

Implementation Step 14

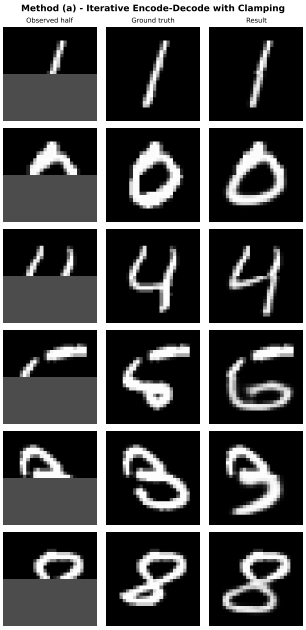


Figure 9.6: Extracted Figure 6

```
model.eval()
N_STEPS_B = 500
sigma2 = 0.01
results_b = []
loss_curves_b = []

for i in range(len(test_imgs)):
    x_obs = test_imgs[i:i+1]

    # Start z from the prior
    z = torch.randn(1, model.latent_dim, device=device, requires_grad=True)
    optimizer_b = optim.Adam([z], lr=0.05)
    losses = []

    for _ in range(N_STEPS_B):

# ... (Code truncated for brevity)
```

Method (c) - Pixel Optimisation via Gradient Descent We keep the entire VAE frozen and directly optimise the missing pixel values. We parameterise the free pixels through a sigmoid so they stay in [0,1]: x_free = sigmoid(y) Then minimise the negative ELBO of the full image with respect to y. Gradients flow back through the VAE into y using the reparameterisation trick.

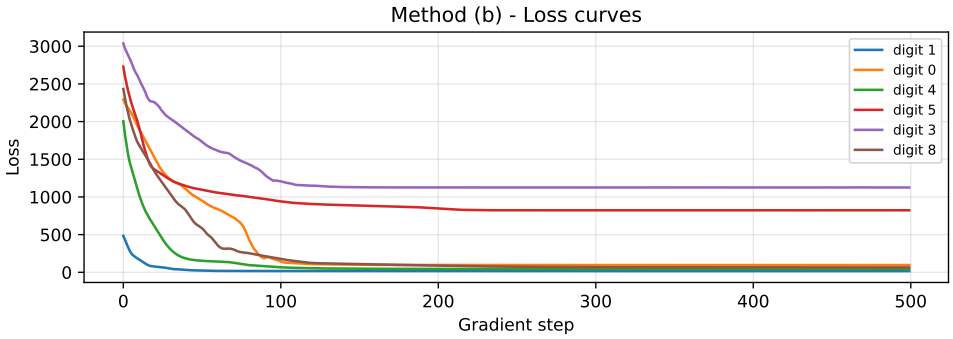


Figure 9.7: Extracted Figure 7

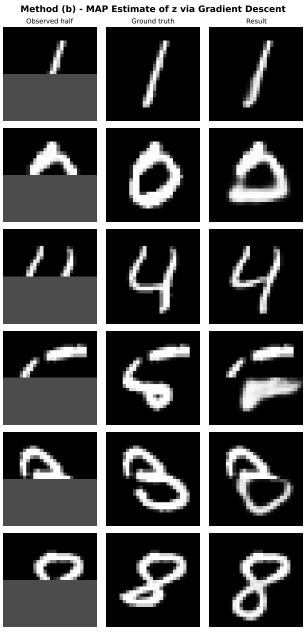


Figure 9.8: Extracted Figure 8

Implementation Step 15

```
model.eval()
N_STEPS_C = 500
results_c = []
loss_curves_c = []

for i in range(len(test_imgs)):
    x_obs = test_imgs[i:i+1]

    # Optimise y in unconstrained space, x_free = sigmoid(y) stays in [0,1]
    y = torch.zeros(1, 1, 28, 28, device=device, requires_grad=True)
    optimizer_c = optim.Adam([y], lr=0.05)
    losses = []
```

```
13
14      for _ in range(N_STEPS_C):
15          optimizer_c.zero_grad()
16
17  # ... (Code truncated for brevity)
```

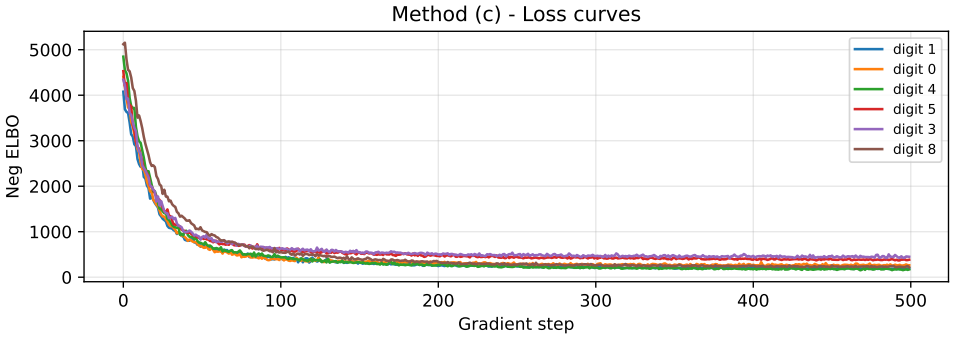


Figure 9.9: Extracted Figure 9

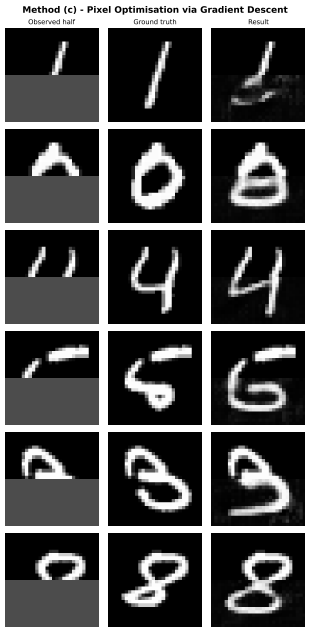


Figure 9.10: Extracted Figure 10

Comparison of the Three Methods **Method (a) - Iterative clamping** works well and is the fastest. The encoder sees the full image at each step, so it uses both the observed pixels and the prior to guide the reconstruction. The result is always a realistic digit because the decoder can only produce things it learned during training. The weakness is that it depends on the random initialisation of the missing pixels. **Method (b) - Optimising z** also produces good results because we search directly in the latent

space, which is smooth and structured. The prior term $||z||^2$ keeps z close to N(0,I) so the decoder always outputs something realistic. The weakness is that gradient descent can get stuck in local minima - the z we find might explain the top half well but not be the most natural completion. **Method (c) - Optimising pixels** is the most principled method mathematically - we minimise the full negative ELBO which uses both the encoder and decoder. However in practice it is the hardest to optimise because we are working directly in pixel space (784 dimensions) which is much harder than latent space (32 dimensions). Some results look slightly blurry or inconsistent because the optimisation can get stuck. **Key insight:** methods that are mathematically more correct do not always give better results in practice. Method (a) is the simplest but often produces the cleanest completions because the encoder naturally constrains the solution to the learned data distribution at every step.